

Fog of War without Zero-Knowledge Proofs: A Blinded Mutual-Sighting Test for Trustless Game Channels

Xaya Developers

Daniel Kraft Andrew Colosimo Konstantin Gorskov
Roy Crombleholme Andrew Gore Johnv5 AI Agent

July 27, 2026

Abstract

Replicated deterministic state machines (blockchains and the state channels built on them) make hidden information awkward: every replica must be able to verify every transition, yet fog of war requires that positions stay secret. Commit–reveal schemes hide positions but destroy *encounters*: two players walk past each other in the dark because neither may learn the other is near. We present a *blinded mutual-sighting test*: after both players have hash-committed their moves for a round, and before either is disclosed, the two clients run a Diffie–Hellman-blinded private membership test (the Meadows 1986 / Huberman–Franklin–Hogg 1999 lineage, cut down from set intersection to a single membership bit evaluated in both directions) over Curve25519 with x-only Elligator 2 hashing. Both parties learn exactly one shared bit (“we are in sight”) and nothing else. Two design choices adapt the classical protocol to a consensus setting: every blinding scalar and pad element is derived deterministically from a seed the player is already bound to reveal, making the entire transcript recomputable by an after-the-fact audit; and no conviction ever trusts the exchange itself, since cheating is adjudicated by recomputing the sight predicate over tiles fixed by prior commitments. Together these replace an in-round zero-knowledge proof, which measurably does not fit our execution budget, with after-the-fact attribution. The system is deployed inside a 78 kB zero-import WebAssembly consensus blob; an independent reimplementations reproduces its transcripts byte for byte. In measured play the test eliminated unnoticed mutual-sight rounds entirely (from 142 to 0) while an optimal Bayesian hunter absorbing every negative answer still faced 211–925 candidate tiles and located a moving player in 0 of 64 matches.

1 Introduction

Multiplayer games on public replicated state machines inherit a contradiction. The chain (or channel) exists so that every participant can verify every transition; fog of war exists so that participants *cannot* see part of the state. The standard resolution, commit now and reveal later, solves only half of the problem. A hash commitment hides a player’s position perfectly, but perfect hiding is precisely what makes the game unplayable: if neither player may learn anything about the other’s position before the endgame, then two players six tiles apart walk past each other in the dark, every round, for the whole match. In our own dungeon-crawler testbed, removing all position disclosure produced 142 rounds of mutual line of sight in which neither player was told anything: an invisible non-game.

The requirement is therefore a conjunction:

1. **Privacy.** Nobody learns anything about your position unless they are close to you (or you spend it deliberately, e.g. on a contested objective).
2. **Encounters.** Two players who *are* close must both learn so, in the same round, every round, with no exceptions.
3. **Adjudication.** Every rule above must remain enforceable by a referee that never sees secret state, on a metered execution budget.

The natural cryptographic answer to requirement 3 is a zero-knowledge proof attached to every move. That answer is unavailable to us in-round: our referee is a zero-import WebAssembly blob metered at 66.3 million units of fuel per call, and a single Groth16 verification measures at 73.6 million. This paper describes what we built instead.

Contributions.

1. A *blinded mutual-sighting test* (§5): a two-party private membership test, run in both directions between move commitment and move disclosure, that yields the one shared bit “ $\text{sight}(a, b)$ ” and nothing else. The construction is the classical DH private-matching protocol of Meadows and of Huberman, Franklin and Hogg [1, 2], specialised from $\text{set} \cap \text{set}$ to membership-of-one-element, instantiated x-only on Curve25519 with an Elligator 2 hash-to-curve.
2. A *determinism-for-auditability* discipline (§7): every byte a party should send is a pure function of a seed it is already committed to reveal, so a judge can recompute any disputed transcript after the fact. Combined with signatures over flights, this converts “prove your input was honest” (a SNARK’s job) into “lying is non-repudiable evidence,” at negligible consensus cost.
3. *Enforcement by geometry, not cryptography* (§7): no conviction ever depends on the exchange. Suppressing a sighting, or disputing a phantom one, are both adjudicated by recomputing the public sight predicate over tiles bound by hash commitments made before the round’s information existed.
4. *Measurements* (§8): the deployed system closes the encounter hole completely (0 unnoticed mutual-sight rounds, previously 142) at ~ 100 ms of curve arithmetic per seat per round; an exact Bayesian attacker fed every negative answer retains 211–925 candidate tiles and located a moving player in 0 of 64 matches. An independent reimplement of every formula in this paper reproduces the deployed blob’s transcripts byte for byte.

2 Setting and threat model

Game channels. The system runs on *game channels* [15] as deployed on the Xaya platform: two players play a turn-based game off-chain, exchanging signed moves, with the blockchain as the fallback arbiter. A single host process referees many different games by loading each game’s rules as a SHA-256-pinned, zero-import WebAssembly blob; the blob is metered (66.3 M fuel per call) and its state capped (8192 bytes). Both clients and every potential arbiter run the identical blob, so “the judge” below means *any* party replaying the rules; honesty of the judge is by replication, not assumption.

Round structure. Play proceeds in rounds on a fixed public map (a 64×64 tile dungeon, procedurally generated from a shared seed). Each round:

1. **Commit.** Both players publish hash commitments to their move (tile and action, salted by a per-match 32-byte secret seed_i).
2. **Sighting test.** The exchange of §5, between the two clients only; the referee is not involved.
3. **Disclose.** If the test said *no*, a player publishes nothing about its position. If it said *yes*, both players are obliged to publish their exact tiles for the round (with an arrival proof; on first contact, the connecting path).

Every match ends with an **endgame audit**: both players reveal seed_i and their full paths, which the blob verifies against every round’s commitments and replays against the map. This audit pre-dates the sighting test; §7 extends it.

Adversary. A malicious client controls all its own messages: it may send arbitrary bytes in the sighting exchange, stall, or disclose falsely. It cannot break hash commitments or signatures, and it cannot avoid the endgame audit except by forfeiting (timeouts convert stalling into forfeit). Both players’ clients may be malicious simultaneously; the game between two colluding cheaters is not our concern, but neither cheater may gain against an honest opponent. The design rule throughout: *consensus never trusts a claim*; everything that convicts is recomputed by the blob from committed data.

3 The sight predicate

Positions are walkable tiles of the public map. For tiles a, b let $d(a, b)$ be the shortest walkable path distance (the implementation caps the search at 9 steps) and let $\text{fovLOS}(a, b)$ be the game’s line-of-sight predicate (a symmetric, corner-aware visibility test on the tile grid). The predicate jointly computed each round is

$$\text{sight}(a, b) := [d(a, b) \leq 6] \wedge \text{fovLOS}(a, b). \quad (1)$$

d is symmetric by construction; fovLOS was verified symmetric exhaustively over all 339 757 in-range walkable pairs of the test maps. Symmetry makes “the” shared bit well defined: with

$$V(x) := \{ t \text{ walkable} : \text{sight}(x, t) \}, \quad a \in V(b) \iff b \in V(a), \quad (2)$$

and $x \in V(x)$ always (distance 0 satisfies (1)). Over 1 116 tiles sampled from 4 generated maps, $|V| \leq 79$; the protocol pads every set to a fixed $N = 128$ (§4.3), so message size and work are constants independent of the sender’s surroundings.

The predicate must be symmetric for the construction to apply at all: asymmetric sight ranges would make the two parties’ bits legitimately disagree and the shared-bit framing collapse. Consequently the game defines exactly one sight range in consensus; light sources are cosmetic.

4 Cryptographic preliminaries

4.1 Group and encodings

We work on Curve25519 [11] in Montgomery form $y^2 = x^3 + Ax^2 + x$ over $\mathbb{F}_p$, $p = 2^{255} - 19$, $A = 486662$, with prime-order subgroup of order

$$\ell = 2^{252} + 27742317777372353535851937790883648493 \quad (3)$$

and cofactor 8. All points are handled x-only (Montgomery u -coordinates) as 32-byte canonical little-endian encodings, fully reduced mod p ; point equality is equality of encodings. Scalar multiplication is the RFC 7748 [13] Montgomery ladder run for a fixed 255 iterations, *without clamping*. Commutativity itself would survive clamping (each clamped scalar is a fixed integer, and integer multiplication commutes); the reason clamping is omitted is auditability: the transcript must be a pure function of the derived scalar with no hidden transformation, so that a judge recomputing a disputed element applies exactly the scalar of §4.3 and nothing else. Small-subgroup safety, clamping’s usual job, is provided instead by cofactor-clearing every point at creation (§4.2). The unclamped ladder core is validated against the RFC 7748 §5.2 known-answer vectors, including the 1 000-fold iterated one, through a clamping test harness.

Because encodings are x-only, P and $-P$ are identified. A false positive in the membership test would therefore require $P_a = \pm P_t$ for distinct tiles; but hash-to-curve outputs *are* x-coordinates, so the “ $-P$ collision” and the “x collision” are the same event, absorbed into the collision bound of Theorem 1.

4.2 Hash-to-curve

H2C(m) is defined by:

1. $u \leftarrow \text{SHA-256}(m)$ read little-endian, top bit cleared, reduced mod p .
2. x-only Elligator 2 [12] with $Z = 2$ (a non-square in $\mathbb{F}_p$ since $p \equiv 5 \pmod{8}$):

$$x_1 = \frac{-A}{1 + Zu^2}, \quad g(x) = x^3 + Ax^2 + x, \quad (4)$$

with the exceptional case $Zu^2 = -1$ mapped to $x = -A$. If $g(x_1)$ is a square in $\mathbb{F}_p$ (Euler’s criterion, one exponentiation), output $x = x_1$; otherwise output $x = -x_1 - A$ ($= x_1 Zu^2$), for which g is then guaranteed square. No y -coordinate is ever computed; the ladder needs only x .

3. Clear the cofactor by multiplying by 8, landing in the prime-order subgroup.

This is a *nonuniform*, single-map, SHA-256-based encoding, not the RFC 9380 [14] suite `curve25519_XMD:SHA-512_ELL2_R0_`. The choice is deliberate. The protocol needs (a) determinism, (b) points with discrete logarithms unknown to everyone, and (c) outputs in the prime-order subgroup. It does not need the map’s output to be indistinguishable from uniform, because every transmitted point is additionally randomised by a per-round blinding scalar; and interoperability is a non-goal, since every party runs the same pinned blob. The test suite checks determinism, pairwise distinctness over sampled inputs, and the identity $\ell \cdot P = \mathcal{O}$ for H2C outputs, the last being the strongest cheap statement, as it proves curve membership and cofactor clearing in one identity (a twist point, or one with an 8-torsion component, fails it).

4.3 Derivations: nothing is random

Every quantity in the exchange is derived, never sampled:

$$\text{tile points (public): } P_t = \text{H2C}(\text{"DCHT"} \parallel \text{LE16}(\text{morton}(t))), \quad (5)$$

$$\text{blinding scalar of player } i, \text{ round } r: \alpha_r^{(i)} = \text{SHA-256}(\text{"DCHP"} \parallel \text{seed}_i \parallel \text{LE32}(r)), \quad (6)$$

$$\text{pad dummies of player } i: D_j^{(i)} = \text{H2C}(\text{"DCHD"} \parallel \text{seed}_i \parallel \text{LE16}(j)), \quad j = |V|, \dots, N-1. \quad (7)$$

The scalar of (6) has its top 4 bits cleared, making it uniform on $[0, 2^{252})$. Since $\ell = 2^{252}(1 + \varepsilon)$ with $\varepsilon \approx 3.8 \cdot 10^{-39}$, the unreachable sliver of $[0, \ell)$ is negligible; and α is a blinding factor, not a signature nonce, so nonce-bias attacks do not apply. The measure-zero case $\alpha = 0$ is patched deterministically. The dummies (7) are intentionally round-independent: per-round freshness comes entirely from α_r , and fixed base points let a client precompute every curve point it will ever hash (the maps are the expensive half of the per-round cost).

seed_i is the same 32-byte secret that already salts player i 's move commitments and is revealed in full at the endgame audit. That reuse is the design (§7): it makes the honest transcript a pure function of data the player is bound to expose.

5 The protocol

Fix round r ; both players have committed (tile, action) for r . Write $\alpha_A = \alpha_r^{(A)}$, and let a, b be the committed tiles. Before any disclosure:

Flight 1 (both directions).

$$A \rightarrow B: \quad S_A = \text{sort}\left(\left\{\alpha_A P_t : t \in V(a)\right\} \cup \left\{\alpha_A D_j^{(A)} : j = |V(a)|, \dots, N-1\right\}\right), \quad Q_A = \alpha_A P_a, \quad (8)$$

and symmetrically $B \rightarrow A: (S_B, Q_B)$. Here S is $N = 128$ points (4096 bytes) plus Q (32 bytes); sort orders by encoding bytes, a canonical order independent of tile enumeration that reveals nothing about which entries are dummies. Exactly $N+1$ hash-to-curve maps and $N+1$ ladder multiplications are performed whatever $|V|$ is, so neither length nor timing leaks the sender's surroundings.

Flight 2 (both directions).

$$A \rightarrow B: \quad R_A = \alpha_A Q_B (= \alpha_A \alpha_B P_b), \quad B \rightarrow A: \quad R_B = \alpha_B Q_A (= \alpha_B \alpha_A P_a). \quad (9)$$

Local finish.

$$\text{bit}_A = \left[R_B \in \{\alpha_A s : s \in S_B\} \right], \quad \text{bit}_B = \left[R_A \in \{\alpha_B s : s \in S_A\} \right], \quad (10)$$

each costing N ladder multiplications and N encoding comparisons.

Theorem 1 (Correctness). *Except with probability negligible in the hash length ($\ll 2^{-100}$ for a 4096-tile universe under SHA-256), $\text{bit}_A = \text{bit}_B = \text{sight}(a, b)$.*

Proof sketch. On the prime-order subgroup, $\alpha_A(\alpha_B P_t) = (\alpha_A \alpha_B \bmod \ell) P_t = \alpha_B(\alpha_A P_t)$, so $R_B = \alpha_A \alpha_B P_a$ equals $\alpha_A(\alpha_B P_t)$ iff $P_a = P_t$ (x-only: iff $P_a = \pm P_t$, the same hash-collision event, cf. §4), i.e. iff $a \in V(b)$, unless two distinct H2C inputs collide, bounded by the birthday term over the $\leq 4096+128$ domain-separated inputs. Thus $\text{bit}_A = [a \in V(b)]$ and symmetrically $\text{bit}_B = [b \in V(a)]$; these coincide with $\text{sight}(a, b)$ by (2). $\square$

The deployed client asserts the two bits agree on every round it can observe both, and treats disagreement as a defect, not an input.

6 Security analysis

Designed leakage: the bit, and (by padding) nothing about $|V|$.

The blinded set hides its contents under DDH. Distinguishing $\{\alpha P_t : t \in T_0\}$ from $\{\alpha P_t : t \in T_1\}$ for candidate tile sets T_0, T_1 , given the public P_t , is a standard DDH-type distinguishing problem in the prime-order subgroup (x-only, exactly as in X25519). The receiver holds nothing else involving α .

Unknown discrete logarithms are load-bearing. The single most instructive pitfall we encountered:

Proposition 1. *If tile points were instead constructed as $P_t = h_t G$ with $h_t = \text{SHA-256}(t)$ public, a receiver of one flight recovers the sender’s entire visible set, hence its position, in $O(|U|)$ scalar multiplications, where $|U|$ is the tile universe.*

Proof. Take two blinded elements $X_1 = \alpha h_{t_1} G$ and $X_2 = \alpha h_{t_2} G$. Every h_v is public and invertible modulo ℓ , so form

$$S_i = \{ h_v^{-1} X_i : v \in U \}, \quad i = 1, 2, \quad (11)$$

at $|U|$ scalar multiplications each. Both sets contain αG , attained at $v = t_i$, and any further coincidence occurs only with negligible probability, so $S_1 \cap S_2 = \{\alpha G\}$ is recovered by hash-table lookup with no hypothesis testing. Given αG , one further pass matching each element of the flight against $\{h_v(\alpha G) : v \in U\}$ labels the whole set. The total is $\approx 3|U|$ scalar multiplications, a few thousand for $|U| \approx 4096$. $\square$

With hash-to-curve, the analogous first step, obtaining αG from αP_t without knowing $\log_G P_t$, is CDH. This is also why brute force over the small tile universe buys nothing: “is this element tile t ?” has no checkable relation without breaking DH. The attack of Proposition 1 is documented in the implementation’s headers so that no future “simplification” can silently reintroduce it.

Cross-round unlinkability. α_r is fresh per round, so linking $\alpha_r P_t$ to $\alpha_{r+1} P_t$ (a stationary player) is again DDH. The dummies being fixed per seed changes nothing: they are re-blinded like every tile.

Reflection and replay. B may echo one of A ’s own set elements back as its Q_B ; then $R_A = \alpha_A^2 P_t$. Exploiting the pair $(\alpha_A P_t, \alpha_A^2 P_t)$ is the square-DH problem; no attack without pairings is known, and Curve25519 is not pairing-friendly. The bit B computes from a garbage Q is garbage (self-sabotage) and becomes attributable evidence under §7 once flights are signed.

Low-order and twist points. A malicious peer can substitute any 32-byte string for a point; the x-only ladder processes off-curve (twist) and low-order inputs without special-casing. The worst case is a small-order point sent as Q : the reply αQ reveals $\alpha \bmod 8$ (the 8-torsion image survives because the scalar is unclamped): at most 3 of α 's 252 bits, useless against DDH. The reply never matches a cofactor-cleared set element, so the resulting bit is 0. Like every malformed flight, self-sabotage now and signed evidence once attribution lands.

The accumulated-negative leak, quantified. Every “no” round tells a player that the opponent is outside its visible set. These constraints accumulate; this is the designed privacy/liveness trade of the whole scheme, and we measured it rather than assuming it away. An exact Bayesian belief-set attacker (forward support propagation, the optimal inference given the protocol's outputs, granted every advantage) leaves a moving player **211–925 candidate tiles** of cover on our maps; active probing strategies located the hider in **0 of 64 matches**. For comparison, the always-on coarse position beacon the test replaced left ~ 71 candidates. These figures must be re-checked if the sight range or movement speeds change.

What the exchange deliberately does not provide. (i) No in-round proof that V was computed from the committed tile: a malicious party can feed any set or Q it likes; the surrounding system makes this pointless or attributable rather than impossible (§7). (ii) No fairness or liveness: a party can stall mid-exchange; the channel's existing timeout forfeits cover it. (iii) It requires the predicate to be symmetric (§3).

7 Attribution instead of zero-knowledge

The referee never verifies the exchange in-round; the happy path costs consensus nothing. Three facts combine into the enforcement story:

1. **The honest transcript is a pure function.** By §4.3, every byte party i should send in round r is $T(\text{seed}_i, r, \text{tile}_i(r), \text{map})$: the scalar from the seed, V from the committed tile and the public map, tile points and dummies from fixed derivations, order fixed by the sort.
2. **The audit reveals the preimages.** The pre-existing endgame audit forces both players to reveal seed_i and their full paths, verified against every round's hash commitments. Any judge can therefore recompute, after the fact, the exact flight bytes an honest party would have sent in any round, and compare with what was sent. With flights ed25519-signed, a wrong byte is non-repudiable evidence against its sender. The recomputation is necessarily of a *single named element*, not of the flight: we measured a full flight rebuild at $\approx 1.98 \times 10^9$ fuel, thirty times the entire per-call cap, whereas verifying a supplied flight against its committed hash and recomputing one element costs a measured 15.9 M, which sits alongside the 17.6 M endgame audit at 51% of the 66.3 M cap that a single Groth16 verification already exceeds at 73.6 M. Binding the bytes therefore uses a commitment the sender already publishes in its own signed move rather than a signature scheme inside the blob, and that commitment is a Merkle root over the flight rather than a flat hash: a dispute then carries one leaf and an eight-step path, roughly 350 bytes, instead of the whole 4,128-byte flight.
3. **Convictions never trust the bit.** “In sight and stayed dark” and “disputed a sighting that was not there” are both adjudicated by recomputing sight (Eq. 1) over tiles bound by

commitments made before the round’s information existed. The bit only tells honest clients when they are obliged to act; the map convicts.

So the SNARK’s job, *prove your input was honest*, is replaced by: *your input is a deterministic function of things you are already bound to reveal, and lying about it is signed evidence*. The guarantee is delivered after the fact instead of in-round, which a turn-based channel game with a mandatory endgame audit can afford; the consensus-side cost is essentially zero.

The trust boundary, stated plainly. Until the signed-flight component is active, transcripts are deterministic but not yet *attributable*: a malicious opponent who feeds a fabricated visible set can make an honest client’s instrument read a phantom “yes,” causing it to over-disclose or to file a dispute it then loses. This framing residual is pinned by a failing-by-design test in the deployed suite and is closed exactly by item 2 above (signatures plus the one-element recompute branch); it is the final component of the design, in progress at the time of writing. Everything else in §6 holds already.

8 Implementation and measurements

The complete rules compile to a single zero-import WebAssembly blob of 78 196 bytes: map generation, movement, combat, commitments, the audit, and the entire sighting test (field arithmetic, ladder, Elligator 2, derivations). The blob is executed identically by both clients (V8) and the metered referee (wasmtime); there is no cryptographic library dependency, and it is self-contained by construction.

Independent verification. Every formula in §§4–5 was reimplemented from this paper’s text alone, in a separate codebase (BigInt arithmetic, no shared code), and compared against the deployed blob: full 4 128-byte flight-1 messages from three positions, flight-2 responses, and the finishing bit are reproduced **byte for byte** (20 checks, 0 failures), alongside the RFC 7748 known-answer vectors through the same core. The prose and the deployed system cannot silently diverge without that harness failing.

Measured behaviour.

universe	64×64 tiles, Morton-indexed; $ V \leq 79$ measured, padded to $N = 128$
flight sizes	flight 1: $128 \cdot 32 + 32 = 4\,128$ B; flight 2: 32 B; ≈ 8.3 kB/round total
per-seat work	$(N+1)$ maps + $(N+1)$ mults (build) + 1 mult (respond) + N mults (finish)
per-seat time	build 71 ms, respond 0.2 ms, finish 29 ms ≈ 100 ms (7% of the 1.4 s beat)
agreement	PET bit vs. referee’s sight: 0 disagreements / 77 live rounds (20 in sight)
encounters	unnoticed mutual-sight rounds: 0 (142 before the test existed)
contested play	met 11/12 matches, decided by combat 11/12, mean first contact round 69
privacy	Bayesian hunter: 211–925 candidate tiles; located in 0/64 matches
ladder KATs	RFC 7748 §5.2, including the 1 000-fold iterated vector

The ~ 100 ms figure is single-threaded on the main thread and halves with precomputation of the fixed points (5), (7), which is why those derivations are round-independent.

9 Related work

The DH private-matching lineage. The double-blinding trick at the core of §5 is old and good. Meadows [1] used commutative blinding for cryptographic matchmaking in 1986; Huberman, Franklin and Hogg [2] used the same DH structure for private community matching in 1999. The modern literature knows it as ECDH-PSI: formalised and extended by Freedman, Nissim and Pinkas [5] and given linear-complexity DH instantiations by De Cristofaro and Tsudik [6]. It runs at planetary scale today: Google’s Password Checkup [8] and Apple’s PSI system [9] are both descendants. Our protocol is this classical construction *reduced*: from $\text{set} \cap \text{set}$ to membership-of-one-element, evaluated in both directions, because the game needs one shared bit, not the overlap.

The implementation we started from. Our engineering entry point into this literature was Thomson’s open-source ECDH-PSI implementation [10], a clear reference implementation of the classical protocol (hash-to-group, blind, exchange, double-blind, with an encrypted-payload layer for revealing matched elements over NIST P-256/secp256k1). Studying and prototyping against it shaped what we kept and what we changed: we replaced the NIST curves with Curve25519 and an explicit Elligator 2 hash-to-curve (Proposition 1 is what happens otherwise for a small universe); dropped the payload-encryption layer entirely (we must never learn *which* element matched, so there is nothing to decrypt); fixed the set cardinality by padding (otherwise $|V|$ leaks for free); and, most importantly, bound the inputs. Like most PSI reference implementations, [10] leaves inputs unbound, so a malicious party can use the protocol as a membership oracle over fabricated sets; our committed-seed determinism and audit (§7) exist precisely to close that gap. We have reported that observation upstream, together with a concrete instance of Proposition 1 in that codebase: its elements are mapped as $h_x G$ with h_x public, and a participant legitimately recovers the peer’s blinding factor times the generator during the exchange, from which it can derive the peer’s payload key for any candidate element and enumerate the peer’s set offline. It is the sharpest illustration we know of why hash-to-curve is not a stylistic detail.

Private equality and proximity testing. Comparing secrets without revealing them goes back at least to Fagin, Naor and Winkler [3]. The closest prior art to our setting is Narayanan et al.’s private proximity testing [7], which reduces location proximity to private equality of grid cells for social location-sharing. Our test differs in the predicate (walkable-path distance plus line of sight on a dungeon map, not Euclidean grid overlap), in being mandatory and bidirectional every round (their users choose when to query; our players are *obliged* to act on the bit, and the obligation is enforced by consensus), and in the adversary handled (their protocols assume honest-but-curious phones; our clients are fully malicious and handled by attribution).

Hidden information in adversarial games. Mental Poker [4] founded the field of playing hidden-information games without a trusted dealer. The modern blockchain line is exemplified by Dark Forest [16], which achieves fog of war on a public chain by attaching a zk-SNARK to every move, proving it valid relative to hidden coordinates. That is the “prove your input was honest” path we could not take: our referee’s fuel cap (66.3 M) is measurably below one Groth16 verification (73.6 M), and channel games settle on chains where verifier costs bind harder still. The determinism-plus-audit construction of §7 is our substitute, available to any turn-based game that already ends in a full reveal. Note also the difference in what is proved: Dark Forest proves

unilateral facts about one player’s hidden move, whereas a sighting is an irreducibly *joint* predicate over two players’ secrets, which is what forces a two-party protocol rather than a proof.

Infrastructure. Game channels are due to Kraft [15], deployed on Xaya, the successor of Huntercoin (to our knowledge the first blockchain to run a multiplayer game world fully on-chain), whose scaling and privacy limits motivated this line of work. Curve25519 is Bernstein’s [11]; the Elligator 2 map is from Bernstein, Hamburg, Krasnova and Lange [12]; the ladder and its test vectors follow RFC 7748 [13]; RFC 9380 [14] is the general hash-to-curve framework from which §4.2 deliberately deviates.

10 Limitations and future work

1. **Attribution is the last unbuilt component.** Signed flights and the one-element recompute branch (§7) are designed and test-pinned but in progress; until they land, fabricated flights can frame an honest disputer, and a conservative client should weigh disputes accordingly.
2. **A one-round rotation edge.** A suppressor whose disclosure happens to be ordered after the honest player’s can dodge a dispute for that single round of fleeting edge-of-range contact; signed transcripts close this fully.
3. **Two seats measured.** The exchange generalises pairwise (6 pairs at 4 seats, cost linear in pairs), but all measurements reported here are two-seat.
4. **Cost.** ~ 100 ms/seat/round of curve arithmetic is acceptable at a 1.4 s beat and halves with precomputation; a worker thread removes UI jank. Sub-100 ms beats would need batching or a faster field implementation.
5. **The leak is a dial, not zero.** The accumulated-negative constraint is inherent to guaranteed encounters: any protocol satisfying requirement 2 of §1 must leak at least the negatives. We chose to measure and publish the resulting anonymity set (211–925 tiles) rather than claim more than the construction gives.

Acknowledgments

Edward Thomson’s open-source PSI implementation [10] was the practical starting point of this work: a readable, working embodiment of the classical protocol that let us prototype in days and decide what a consensus setting had to change. The protocol itself stems from four decades of prior art beginning with Meadows [1]; our contribution is the adaptation, not the trick.

References

- [1] C. Meadows. A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party. In *IEEE Symposium on Security and Privacy*, 1986.
- [2] B. A. Huberman, M. Franklin, and T. Hogg. Enhancing privacy and trust in electronic communities. In *ACM Conference on Electronic Commerce (EC)*, 1999.

- [3] R. Fagin, M. Naor, and P. Winkler. Comparing information without leaking it. *Communications of the ACM*, 39(5), 1996.
- [4] A. Shamir, R. L. Rivest, and L. M. Adleman. Mental poker. Technical Report MIT-LCS-TM-125, Massachusetts Institute of Technology, 1979.
- [5] M. J. Freedman, K. Nissim, and B. Pinkas. Efficient private matching and set intersection. In *EUROCRYPT*, 2004.
- [6] E. De Cristofaro and G. Tsudik. Practical private set intersection protocols with linear complexity. In *Financial Cryptography and Data Security*, 2010.
- [7] A. Narayanan, N. Thiagarajan, M. Lakhani, M. Hamburg, and D. Boneh. Location privacy via private proximity testing. In *Network and Distributed System Security Symposium (NDSS)*, 2011.
- [8] K. Thomas et al. Protecting accounts from credential stuffing with password breach alerting. In *USENIX Security Symposium*, 2019.
- [9] A. Bhowmick, D. Boneh, S. Myers, K. Talwar, and K. Tarbe. The Apple PSI system. Apple Inc. technical report, 2021.
- [10] E. A. Thomson. Private-Set-Intersection: an ECDH-based PSI implementation. <https://github.com/EdwardAThompson/Private-Set-Intersection>, accessed July 2026.
- [11] D. J. Bernstein. Curve25519: new Diffie–Hellman speed records. In *Public Key Cryptography (PKC)*, 2006.
- [12] D. J. Bernstein, M. Hamburg, A. Krasnova, and T. Lange. Elligator: elliptic-curve points indistinguishable from uniform random strings. In *ACM Conference on Computer and Communications Security (CCS)*, 2013.
- [13] A. Langley, M. Hamburg, and S. Turner. Elliptic curves for security. RFC 7748, 2016.
- [14] A. Faz-Hernández, S. Scott, N. Sullivan, R. S. Wahby, and C. A. Wood. Hashing to elliptic curves. RFC 9380, 2023.
- [15] D. Kraft. Game channels for trustless off-chain interactions in decentralized virtual worlds. *Ledger*, 1:84–98, 2016. <https://doi.org/10.5195/ledger.2016.15>.
- [16] Dark Forest team. Dark Forest: a zkSNARK-based incomplete-information game on Ethereum. <https://zkga.me>, 2020.